

Privacy-Preserving Source Code Vulnerability Detection & Repair

Retrieval-Augmented LLMs for Visual Studio Code

PRESENTED BY

Md Hafizur Rahman

Mat. No. 810641 · M.Sc. Web Engineering

SUPERVISOR

Abubaker Gaber (Internal)

EXAMINER

Dr.-Ing. Sebastian Heil

DATE

June 16, 2026

The Developer's Dilemma



Can't use the cloud

Sensitive code can't be pasted into a cloud model without breaking compliance.



Scanners miss bugs

The local scanner keeps missing real vulnerabilities, so the bug reaches production.



Warnings, no fixes

When a scanner flags an issue it only warns. Writing the repair is still your job.



What's the alternative?

No tool today combines privacy, full coverage, and one-click repair. Closing that gap is what this thesis sets out to do.

Today You Pick: Send Code Away, or Get No Help



Cloud LLM (Copilot, ChatGPT, Cursor)

Source code crosses the network on every request.

Blocked outright in finance, healthcare, defence,
and air-gapped environments.

Strong reasoning • zero privacy guarantees.



Local SAST (Semgrep, CodeQL, ESLint)

Stays local. Cannot reason across files or generate fixes.

Semgrep canonical recall on our JS/TS corpus: 12.68 %

Strong privacy • poor coverage • no repair.

Either choice forces a trade-off: privacy or capability, never both.

No Tool Combines All Three



Local Inference

Code never leaves the developer's machine.
Verifiable network boundary.



Retrieval-Augmented

Grounded in CWE / OWASP knowledge.
Not just pattern matching.



Developer-Controlled Repair

Structured fix returned.
Developer applies via Quick Fix.
No silent rewrites.

This thesis builds and evaluates a tool that delivers all three.

Research Questions and Requirements

RESEARCH QUESTIONS

- RQ1** Can a local LLM on consumer hardware reach acceptable detection quality under a strict privacy envelope?
- RQ2** Does grounding the LLM in retrieved CWE / OWASP knowledge (RAG) improve detection?
- RQ3** Is in-IDE latency acceptable for developer workflows?

FIVE REQUIREMENTS

- | | | | |
|-----------|--|-----------|--|
| R1 | Accuracy
precision, recall, F1 | R2 | Consistency
JSON parse, inter-run |
| R3 | Repair Quality
manual + auto-applicable | R4 | Usability
latency, IDE integration |
| R5 | Privacy
zero exfiltration, signed corpus | | |

R1-R5 map to fixed threshold scales, set before evaluation, with no post-hoc bands.

RAG = look up known CWE / OWASP weakness patterns before answering · CWE = standard catalogue of software-weakness types
F1 = balance of catching real bugs vs. false alarms (higher = better) · FPR = share of safe code wrongly flagged (lower = better)

Three Existing Approaches

CLOUD LLM ASSISTANTS

GitHub Copilot

Microsoft · 2021

ChatGPT / GPT-4

OpenAI · 2022

Cursor

Anysphere · 2023

Claude Code

Anthropic · 2024

LOCAL SAST

Semgrep

Rule-based · OSS

CodeQL

GitHub · taint analysis

ESLint security plugins

Lint-level

Bandit / SpotBugs

Other languages

ACADEMIC LLM-FOR-SEC

IRIS + GPT-4

Li et al. · ICLR 2025

SecRepair

Islam et al. · NDSS 2024

LLMSecGuard

Kavian et al. · EASE 2024

RESCUE

Shi & Zhang · 2025

No Tool Satisfies R3 + R4 + R5 Together

Solution / Approach	R1 ACCURACY	R2 CONSISTENCY	R3 REPAIR	R4 USABILITY	R5 PRIVACY
GitHub Copilot	✓	✓	✓	—	✗
Semgrep	—	✓	✗	✓	✓
CodeQL	—	✓	✗	—	✓
IRIS (GPT-4)	—	—	✗	✗	✗
SecRepair	—	—	—	✗	—

LEGEND

✓

High / Satisfied

—

Partial

✗

Not satisfied

THE GAP

No existing tool delivers
repair + IDE-grade
latency
+ privacy together.

Levels apply the fixed R1–R5 scales, set before evaluation: accuracy / repair thresholds [Johnson 2013; Monperrus 2014], latency [Card 1983; Nielsen 1994], privacy = checklist. Full scale + sources in Appendix A8.

Proposed Workflow



Real-time Diagnostics

Inline squiggles per function as you save.



On-demand File Scan

Right-click → scan the current file.



Interactive Q&A

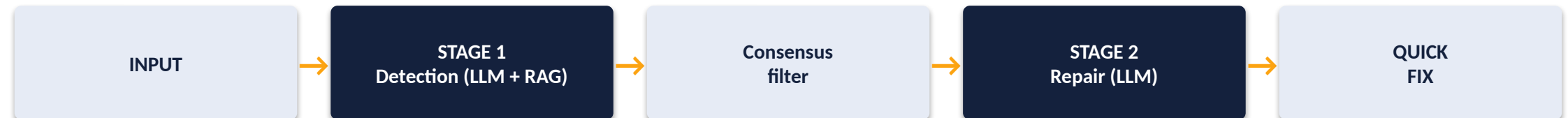
Ask about a finding in a side panel.



Workspace Audit

Batch scan the whole project.

ALL FOUR WORKFLOWS HIT THE SAME PIPELINE



Two-Stage Local Pipeline



Stage 1 • Detection

- JSON-mode LLM call • schema-pinned output
- Optional RAG context • top-k CWE / OWASP snippets
- Two consensus passes • distinct seeds (42, 137)
- Finding kept only when both passes agree
- Returns: category, severity, line range, evidence



Stage 2 • Repair

- Separate structured call • invoked only when detected
- Returns { code, language } • no-fix return allowed (18 / 297)
- Validator parses output with @babel/parser • rejects prose
- Surfaced as a VS Code Quick Fix
- Developer applies the patch • no silent rewrites

Privacy by construction • Ollama bound to loopback • Embeddings local • No telemetry

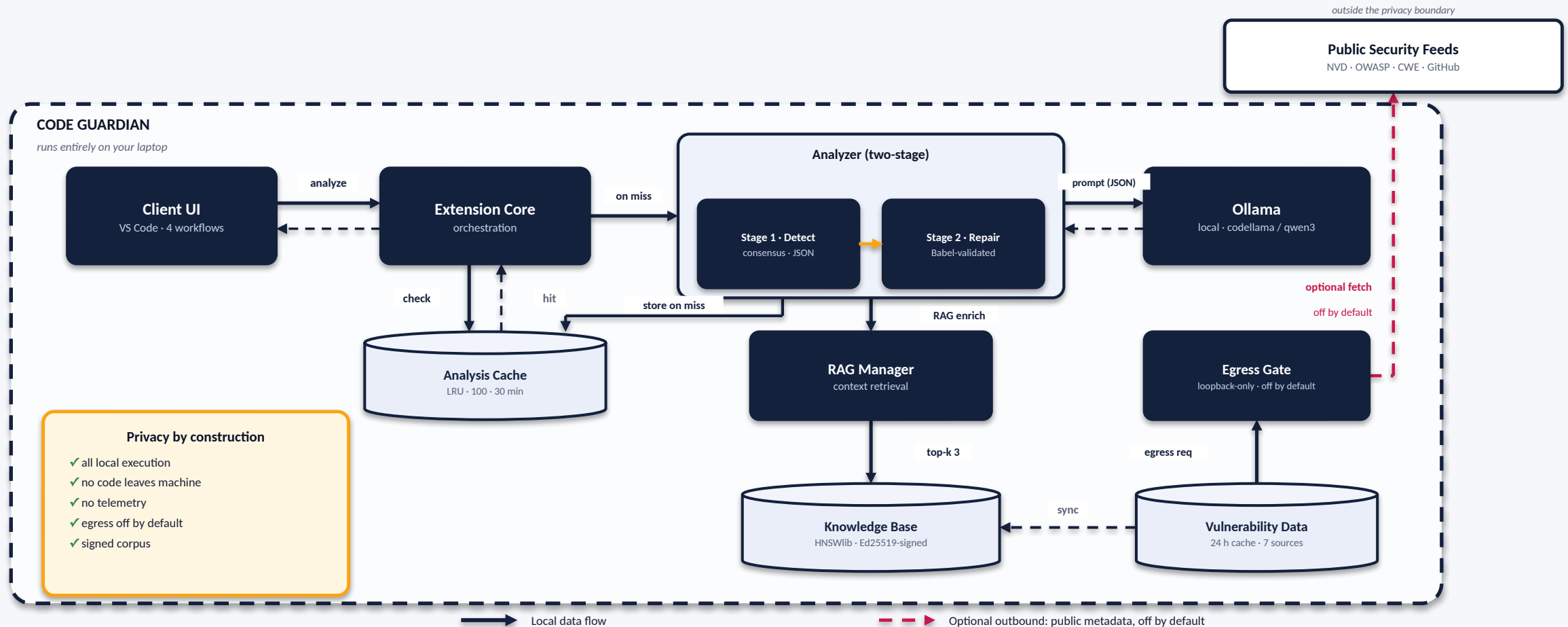
Live in VS Code

```
# test-sample.js > ...
28 app.use((req, res, next) => {
31   next();
32 });
33
34 // CWE-798 + CWE-614/1004: hardcoded session secret, cookie not HttpOnly/Secure
35 // so the session id is readable by JS and sent over plaintext HTTP.
36 app.use(
37   session({
38     secret: "keyboard cat",
39     resave: false,
40     saveUninitialized: true,
41     cookie: { httpOnly: false, secure: false },
42   })
43 );
44
45 // CWE-347: token is DECODED, never verified – any forged JWT is trusted.
46 // Complexity is 4 Everything is cool!
47 app.get("/me", (req, res) => {
48   const token = (req.headers.authorization || "").replace("Bearer ", "");
49   const claims = jwt.decode(token);
50   res.json({ user: claims && claims.sub, admin: claims && claims.admin });
51 });
52
53 // CWE-347: algorithm confusion – accepting "none" and mixing HMAC/RSA let: an
54 // attacker present an unsigned or HMAC-signed token verified with the public key.
55 app.get("/admin", (req, res) => {
56   const token = (req.headers.authorization || "").replace("Bearer ", "");
57   const claims = jwt.verify(token, JWT_SECRET, {
58     algorithms: ["HS256", "RS256", "none"],
59   });
60   res.json({ ok: claims.admin === true });
61 });
62
63 // CWE-208: non-constant-time comparison of a secret leaks it via timing.
64 function checkApiKey(provided) {
65   return provided === API_KEY;
66 }
67
68 app.get("/keycheck", (req, res) => {
69   res.json({ valid: checkApiKey(req.query.key) });
70 });
71
72 // CWE-943: NoSQL injection – operator objects in the body (e.g.
```



SCAN • Install from VS Code Marketplace

System Architecture



Privacy by Construction



Loopback-only Ollama

Bound to 127.0.0.1. Network isolation arm of the threat model, enforced by network policy.



Signed RAG Corpus

Ed25519 manifest verified at load. Provenance arm of the threat model. The corpus cannot be silently rewritten.



Prompt-Injection Harness

12 cases of crafted prompts attempting exfiltration. leakFreeRate = 100 %.



Pinned Container

Node 20.19.0-alpine · npm ci · seed = 42 · byte-identical runs.

Evaluation Setup



Corpus

Curated JS / TS · 101 cases

(71 vulnerable + 30 secure)

20 CWE categories

External: 15 cases from NodeGoat,

Juice Shop, 3 named CVEs

Whole-project scan: OWASP NodeGoat



Configurations

5 Ollama models × {LLM-only, LLM+RAG}

= 10 configurations

+ 3 SAST baselines

(Semgrep, CodeQL, ESLint)

3 runs per sample · deterministic

→ 303 invocations per config



Statistical Discipline

Held-out 71 / 30 test split
(threshold tuning off the test set)

McNemar with Bonferroni
across paired model comparisons

Exact-binomial CIs for small-n rates

Results

R1 • DETECTION

MEDIUM

71.43 %

F1 · qwen3:8b + RAG · P 72.46 % · R 70.42 %
On unseen code: held-out 61.11 % · external 63.64 %

R3 • REPAIR

HIGH

90.32 %

Auto-applicable: parses & applies (n = 279)
Semantic-correct ≈ 68 % combined · 42 % run as-is (n = 25)

R4 • LATENCY

HIGH

2.2 s

Median stage-1 · p95 = 4.3 s
Comfortably inside on-demand band (≤ 5 s)

R5 • PRIVACY

MET

100 %

leakFreeRate · 12-case prompt-injection harness
schemaValidRate 83 % (10/12) · loopback-only by policy

R2 • HIGH

Consistency: 100 % JSON-parse success, 3,030 invocations (inter-run agreement is deterministic by seed). Colour = requirement band (A8). R1 / R3 / R4 / R5 detailed next.

R1 • Detection Accuracy

qwen3:8b + RAG • canonical taxonomy • full 101-case run

71.43 %

F1

72.46 %

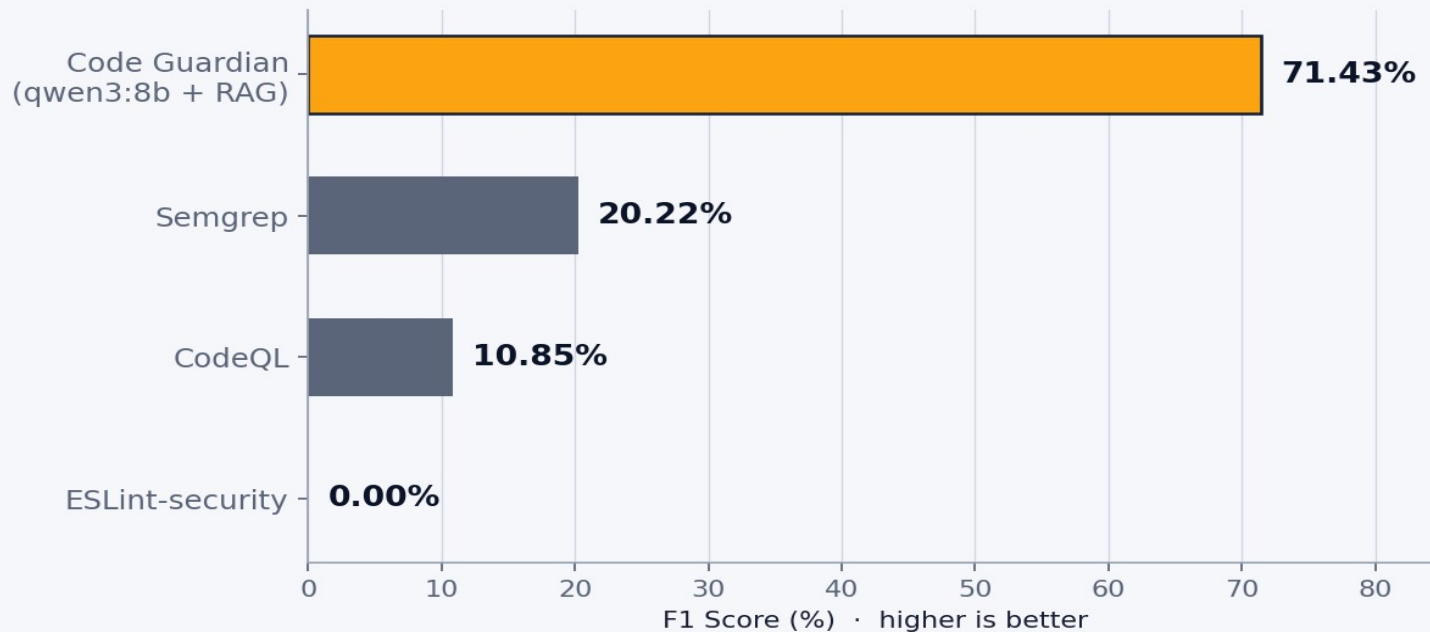
Precision

70.42 %

Recall

10.0 %

FPR



On unseen code

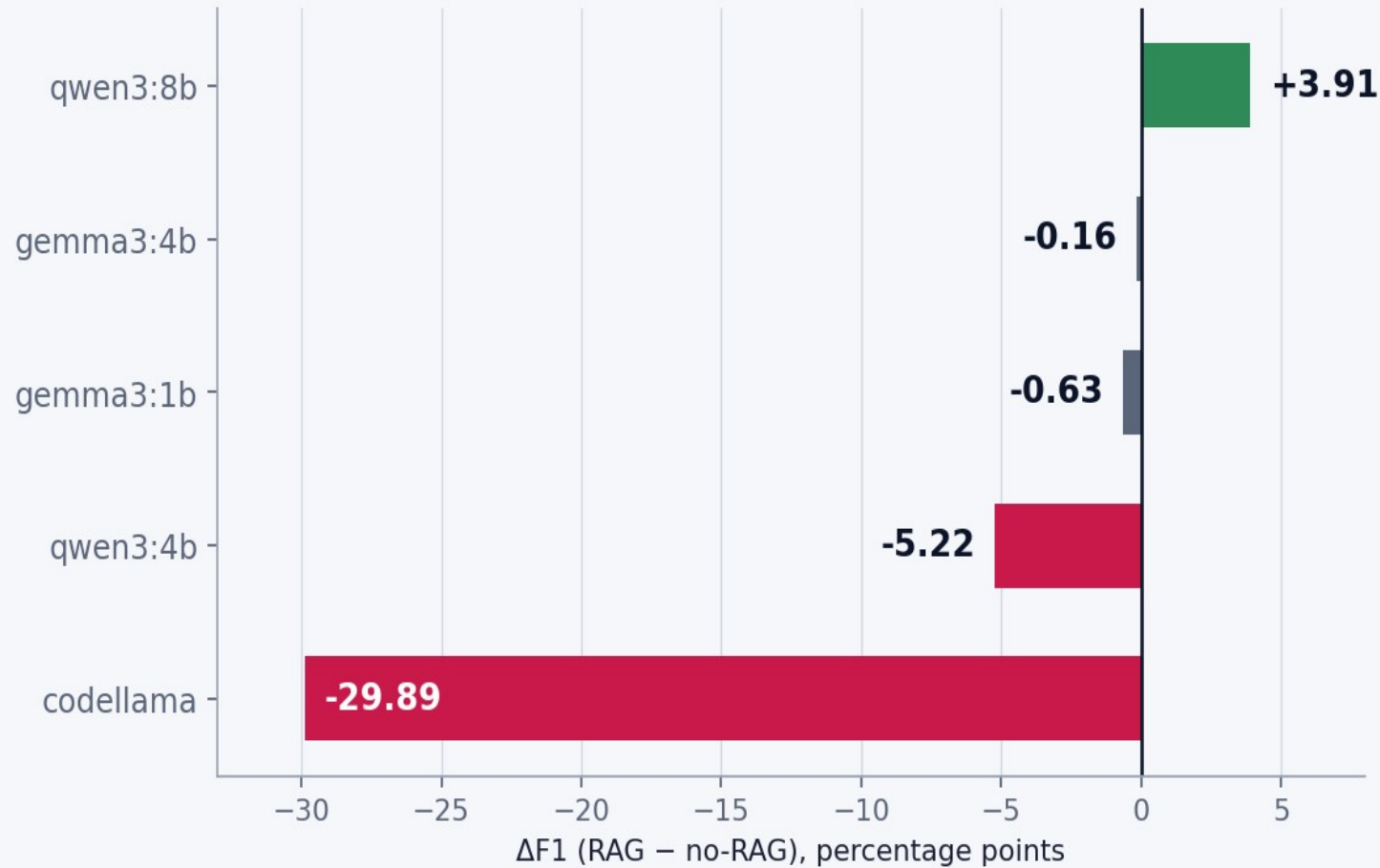
Held-out split: F1 61.11 %
External corpus: F1 63.64 %
→ recall gives ground, precision holds

FAIRNESS

FPR 10 % · 95 % CI [2.1–26.5 %]
SAST shown at function scope,
understating full-project recall (A7)

Optional ≥ 0.67 rescore gate → A9

RAG Is Not a Uniform Win



TAKEAWAY

**RAG can help, harm,
or do nothing,
depending on the model.**

Only codellama's -29.89 F1 survives Bonferroni at $\alpha = 0.01$ (McNemar $p = 0.0013$).

Design implication: treat RAG as a per-model configuration choice, not a default.

R3 • Repair Quality

AUTO-APPLICABLE

Validator parses with @babel/parser • fully automated

On issued fixes (252 / 279)

90.32 %

Across all stage-2 calls

84.85 %

MANUAL REVIEW • n = 25

Rubric-based review on a sampled subset

Fix provided

76 %

Semantically correct

89.5 %

Strategy-aligned

94.7 %

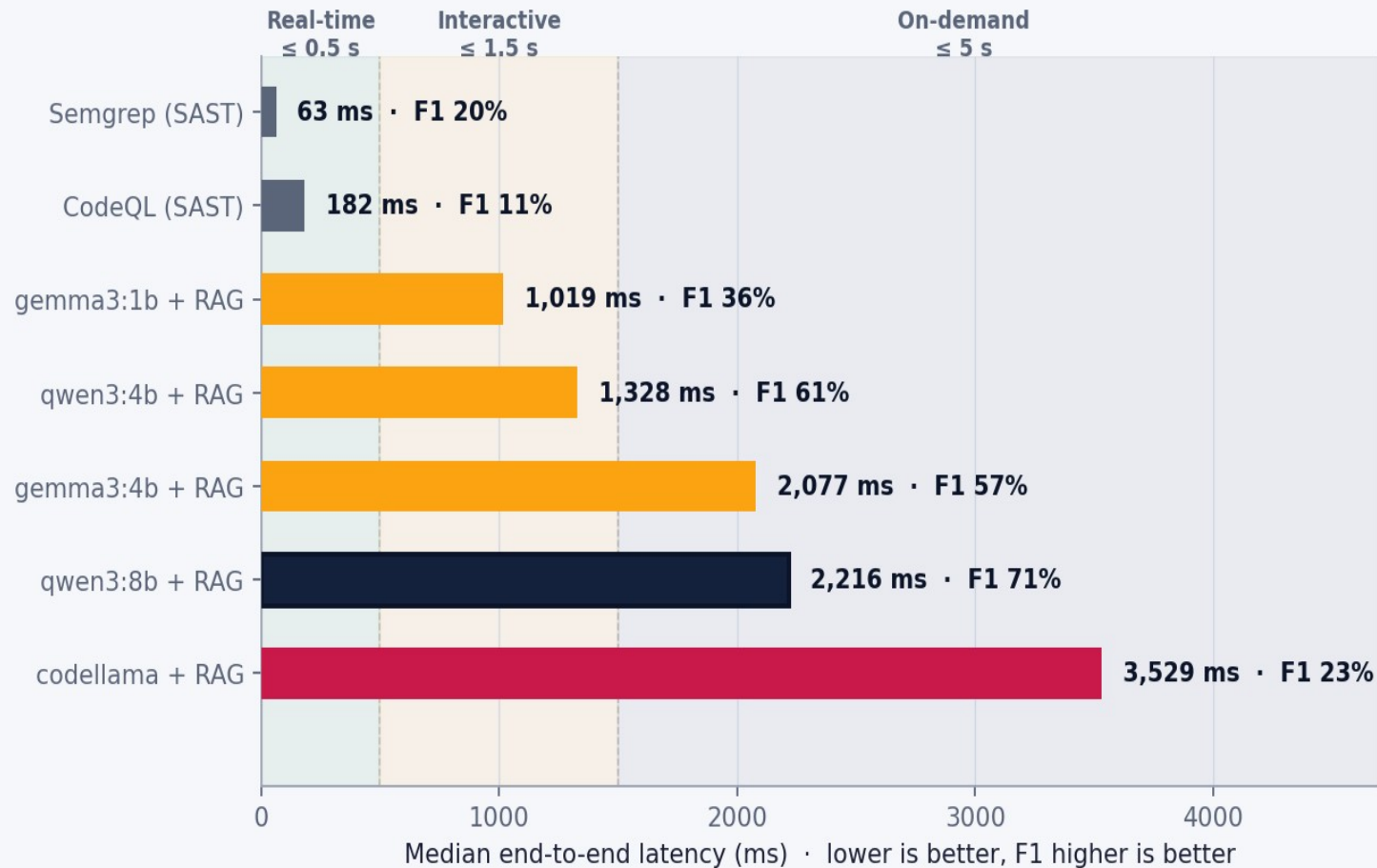
Executable code

42.1 %

TWO-TIER METRIC

Auto-applicable = deployment reading. Manual = decision-support reading. Combined = $76 \% \text{ fix-rate} \times 89.5 \% \text{ semantic} \approx 68 \%$. Stage 2 returns { code, language }; 18 of 297 calls returned no fix.

R4 • Usability: Latency Bands



Real-time ≤ 500 ms

SAST baselines only



Interactive ≤ 1.5 s

gemma3:1b + RAG (1,019 ms)
qwen3:4b (1,328 ms)



On-demand ≤ 5 s

qwen3:8b + RAG (2,216 ms) ← headline
p95 = 4,327 ms

R5 • Privacy: Empirically Verified



Network Isolation

Loopback-only by network policy. Source-leak scan: 0 non-loopback refs in 9,488 code windows.

100 % satisfied



Prompt-Injection Harness

12 crafted exfiltration attempts: no code echoed • schemaValidRate 83 % (10/12).

100 % leakFreeRate



Ed25519-Signed Corpus

RAG corpus provenance verified at load (manifest signature checked).

Verified ✓



Reproducible Build

Node 20.19.0-alpine • npm ci • seed = 42
byte-identical runs.

Reproducible ✓

Resource footprint: harness 3 ms median CPU, under 100 MB RSS • Ollama 6–8 GB RAM for 8B-q4.

What the Evaluation Showed



A local 8B model is enough • RQ1

qwen3:8b + RAG reaches ~71 % F1 on JS/TS, well inside the on-demand latency band on a developer laptop. The privacy envelope holds without paying for it in quality.



RAG is model-dependent • RQ2

The largest paired effect in the dataset is a RAG-induced degradation (codellama -29.89 F1). Treat retrieval as a per-model design choice, not a default.



Repair is the real product gap

90 % auto-applicable means the developer can act, not just be warned. This is where SAST falls flat: they detect, they don't fix.



Privacy as a verifiable boundary

Loopback + signed corpus + container pin makes privacy testable, not aspirational. leakFreeRate = 100 % isn't a slogan; it's a measurement.

RQ1 Yes: 71.43 % F1 under the privacy envelope • RQ2 Yes, but per-model: RAG is not a default • RQ3 Yes: 2.2 s median, inside the on-demand band

Code Guardian Closes the Gap

Solution / Approach	R1 ACCURACY	R2 CONSISTENCY	R3 REPAIR	R4 USABILITY	R5 PRIVACY
GitHub Copilot	✓	✓	✓	—	✗
Semgrep	—	✓	✗	✓	✓
CodeQL	—	✓	✗	—	✓
IRIS (GPT-4)	—	—	✗	✗	✗
SecRepair	—	—	—	✗	—
Code Guardian	—	✓	✓	✓	✓

LEGEND

- ✓ High / Satisfied
- Partial
- ✗ Not satisfied

GAP CLOSED

The only approach to bring privacy + repair + IDE-grade latency together in one local tool, with competitive detection accuracy.

Green check = meets the pre-registered threshold; amber = Medium. R1 is amber: F1 71 % lands in the Medium band, competitive but not High.

Limitations & Future Work

Current Limitations



Single-reviewer manual review (n = 25)

Headline R3 is the objective auto-applicable rate; inter-rater κ is future work.



Curated 101-case corpus

Offset by held-out 71 / 30 split, 15-case external set, whole-project NodeGoat run.



One real-world project end-to-end

NodeGoat only · multi-project validation pending.



Consensus filter inert under determinism

Three seed-42 runs are byte-identical; a real signal needs seed rotation.



No cloud-LLM baseline

Excluded by the privacy threat model · 3 SAST tools set the floor.

Future Work



Seed rotation

Genuine consensus signal beyond the deterministic-decoding floor.



Inter-rater agreement

Second reviewer on the 25-sample subset for a κ statistic.



Multi-project real-world validation

Measured recall beyond NodeGoat, across diverse production codebases.



Streaming partial results

Surface first-token findings to break into the real-time band.



Adaptive RAG gating

Per-model RAG enable/disable driven by slide 16's finding.

Contributions of This Thesis



Working VS Code extension

Local LLM + RAG + signed corpus + pinned container, published to the Marketplace.



Two-stage pipeline with structured contracts

Detection (JSON-mode) + Repair ({ code, language }), schema-pinned at every boundary.



Empirical study

5 models $\times$ 2 modes + 3 SAST baselines $\cdot$ 303 invocations per config $\cdot$ CWE-mapped JS/TS corpus.



Reproducibility infrastructure

Byte-identical runs, Ed25519-signed manifest, pinned Node 20.19.0-alpine container.



Statistical discipline

Bonferroni-corrected McNemar, exact-binomial CIs, held-out 71 / 30 split.

References

[1] Touvron, H., et al.: Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288* (2023)

[2] Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS* (2020)

[3] Li, Z., et al.: IRIS: Using LLMs for context-aware vulnerability detection. *ICLR* (2025)

[4] Islam, N., et al.: SecRepair: LLM-driven security vulnerability repair. *NDSS* (2024)

[5] Kavian, S., et al.: LLMSecGuard: Securing LLM-generated code. *EASE* (2024)

[6] Peng, B., et al.: CWEval: Outcome-driven evaluation of LLM code generation. *ACL* (2025)

[7] Pearce, H., et al.: SecurityEval dataset: mining vulnerability examples. *MSR4P&S* (2022)

[8] MITRE: Common Weakness Enumeration (CWE) · <https://cwe.mitre.org>

[9] OWASP Foundation: OWASP Top 10 · <https://owasp.org/Top10>

[10] Malfa, E.L., Petrov, A., et al.: A taxonomy of evaluation methods for LLMs. *arXiv:2402.13887* (2024)

[11] Ollama: Local LLM serving · <https://ollama.com>

[12] Microsoft: VS Code Extension API · <https://code.visualstudio.com/api>

Thank You

Questions & Discussion



HEADLINE

F1 71.43 % • Repair 90.32 % auto-applicable • Latency 2.2 s median • leakFreeRate 100 %

A1 • Full Per-Model Results • Canonical Taxonomy

Configuration	F1	Prec	Recall	FPR	Latency p50	VRAM
qwen3:8b + RAG	71.43	72.46	70.42	10.00	2,216 ms	~6 GB
qwen3:8b (no RAG)	67.52	61.63	74.65	50.00	1,534 ms	~6 GB
qwen3:4b + RAG	60.92	51.46	74.65	100.00	1,328 ms	~4 GB
qwen3:4b (no RAG)	66.14	56.95	78.87	90.00	1,305 ms	~4 GB
gemma3:4b + RAG	57.14	46.85	73.24	100.00	2,077 ms	~4 GB
gemma3:4b (no RAG)	57.30	46.49	74.65	100.00	2,550 ms	~4 GB
gemma3:1b + RAG	36.02	32.22	40.85	23.33	1,019 ms	~2 GB
gemma3:1b (no RAG)	36.65	29.17	49.30	53.33	1,215 ms	~2 GB
codellama + RAG	23.17	14.79	53.52	100.00	3,529 ms	~6 GB
codellama (no RAG)	53.06	41.60	73.24	100.00	2,024 ms	~6 GB
Semgrep (SAST)	20.22	50.00	12.68	6.67	63 ms	n/a
CodeQL (SAST)	10.85	12.07	9.86	53.33	182 ms	n/a
ESLint-security	0.00	0.00	0.00	0.00	1 ms	n/a

qwen3:8b + RAG headline uses phase1-b3 canonical run; all others use canonical-taxonomy ablation. p95 latencies in source JSON.

A2 • Confusion Matrices • qwen3:8b + RAG

Full corpus (n = 101) • case-level majority

	Predicted: Vuln	Predicted: Safe
Actual Vuln	50 TP	21 FN
Actual Safe	3 FP	27 TN

Held-out test split (n = 30)

	Predicted: Vuln	Predicted: Safe
Actual Vuln	11 TP	10 FN
Actual Safe	0 FP	9 TN

Sample-level via $\geq 2/3$ majority over 3 deterministic runs (seed=42, T=0). Precision is finding-level/pooled, so 72.46 % / $TP/(TP+FP)$ here. Source: phase1-b3 canonical taxonomy.

A3 • Why the Single-Reviewer Manual Review Is Bounded

Fix provided

76 %

Of 25 reviewed samples, 19 included a repair suggestion.

Semantically correct

89.5 %

Fix addresses the CWE (17 of 19 issued fixes).

Strategy-aligned

94.7 %

Right repair pattern (parameterized query, escaping, etc.).

Executable code

42.1 %

Patch runs without modification; the rest give prose guidance.

Inter-rater on the 25-sample subset = future work. Headline R3 is the fully objective auto-applicable rate (90.32 %, n = 279).

A4 • Why a Curated Corpus, Not an Existing Benchmark

WHY EXISTING JS/TS BENCHMARKS DON'T FIT

Juliet (NIST)

C / C++ / Java only. No first-party JS port.

OWASP Benchmark

Java only.

SecurityEval (Pearce 2022)

Small, partially synthetic. CWE labelling inconsistent with OWASP Top 10.

CyberSecEval

29.3 % of vulnerable samples are reproducible (Peng et al. 2025).

CWEval (Peng 2025)

119 curated tasks across 31 CWEs, 5 languages, peer-comparable to 101 cases.

SELECTION-BIAS MITIGATIONS IN THIS THESIS

Held-out 71 / 30 split

Frozen TEST set keeps threshold tuning out of headline numbers.

15-case external corpus

NodeGoat, Juice Shop, 3 named CVEs, independent of the curator.

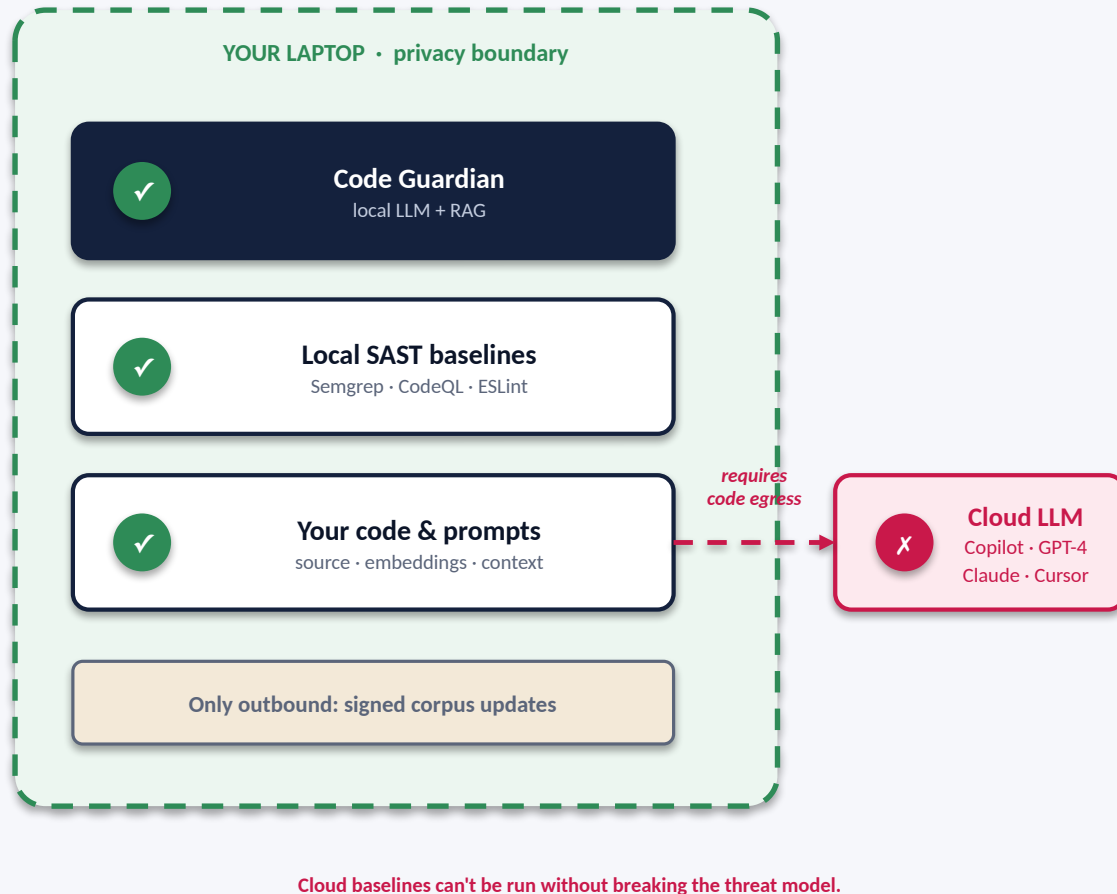
Whole-project NodeGoat run

Project-level recall independent of sample selection.

Provenance bound into manifest

Ed25519 signature; corpus cannot be silently rewritten.

A5 • Why No Cloud-LLM Baseline



ARGUMENT

Cloud-LLM is the problem statement

Including it as a baseline contradicts the threat model.

Floor provided by 3 SAST baselines

Semgrep, CodeQL, ESLint • all local, all reproducible.

Frontier cloud will always win raw F1

The thesis competes on the constraint set: local, zero-egress, reproducible.

Post-defense addendum

A one-paragraph comparison to published GPT-4 numbers on similar JS corpora is straightforward.

A6 • Why R2 Reports 100 % Inter-Run Agreement

RUN 1

```
{
  "findings": [
    {
      "category":
        "sql-injection",
      "severity":
        "high",
      "line": 42
    }
  ]
}
```

RUN 2

```
{
  "findings": [
    {
      "category":
        "sql-injection",
      "severity":
        "high",
      "line": 42
    }
  ]
}
```

RUN 3

```
{
  "findings": [
    {
      "category":
        "sql-injection",
      "severity":
        "high",
      "line": 42
    }
  ]
}
```

WHY THEY'RE IDENTICAL

temperature = 0 • seed = 42 • format = JSON • fixed prompt. Byte-identical → consensus signal requires seed rotation (future work).

A7 • SAST Baseline Configuration

Semgrep**20.22 %**

Registry packs: p/security-audit · p/javascript ·
p/typescript · p/owasp-top-ten (--metrics=off)

CodeQL**10.85 %**

Suite: javascript-security-and-quality.qls
JavaScript database, default queries

ESLint**0.00 %**

eslint-plugin-security v4 · recommended config
(eslint ^9.21); no rule fired on the corpus

FAIRNESS CAVEAT

All three were run per-snippet, at the same function-level scope as the LLM, the only scope-parity setting. In full-project deployment they get cross-file data flow, so these recall figures understate their production performance. Read as “SAST under LLM-equivalent scope.”

A8 • Requirement Scales: Metrics, Thresholds & Sources

Requirement	Metric(s)	Levels & thresholds	Source
R1 • Accuracy	F1, precision, recall, FPR	High: $F1 \geq .75$, $P/R \geq .70$, $FPR \leq .20$ • Medium: $F1 \geq .60$, $FPR \leq .40$ • Low: $F1 \geq .45$ • Insufficient: $F1 < .45$ or $FPR > .70$	Johnson 2013; Ji 2023
R2 • Consistency	JSON parse rate, inter-run agreement	High: $\text{parse} \geq 99\%$, $\text{agreement} \geq 90\%$ • Medium: 95–99% • Low: 85–95% • None: <85%	Harness (raw agreement)
R3 • Repair	Auto-applicable parse rate + manual rubric	High $\geq 75\%$ • Medium 50–75% • Low <50% (3 bands; repair quality is partly qualitative)	Monperrus 2014
R4 • Usability	End-to-end median latency	High: ≤ 5 s inline / ≤ 15 s full-file • Medium: 5–15 s • Low: >15 s	Card 1983; Nielsen 1994
R5 • Privacy	5-item verification checklist	Binary: all five criteria pass, or the requirement fails (privacy is a design constraint, not a spectrum)	Design constraint (Ch. 2)

WHY THESE THRESHOLDS

Fixed in Chapter 2 before any model ran, the same pre-registration discipline as the held-out split. Accuracy and repair bands follow IR/SE convention; latency bands follow HCI response-time limits; privacy is a binary design constraint.

A9 • Confidence Gate: Eval-Only Rescore Filter

Metric	Ungated	Gate ≥ 0.67	
F1	71.43 %	74.07 %	▲ precision-driven
Precision	72.46 %	78.13 %	▲ +5.67 pts
Recall	70.42 %	70.42 %	= unchanged
FPR	10.0 %	10.0 %	= unchanged

WHAT THE GATE ACTUALLY DOES

- Drops findings the model could not map to a specific canonical category (the low-signal 'other' bucket): 6 findings in this run, all false positives. Recall is untouched.
- NOT run consensus: the three evaluation runs share seed 42 and are byte-identical, so there is no run-to-run disagreement to gate on, so the gate reduces to a category filter.
- Off by default in the shipped extension (minConfidence). Both gated and ungated numbers are reported in the thesis, so the reader picks the bar.